

Laboratorio di Calcolo

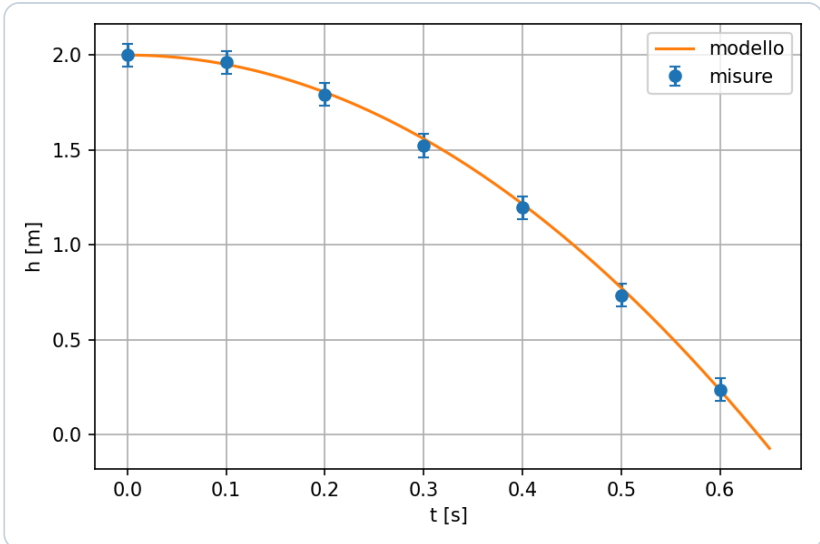
A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Grafici con Python

Perché fare grafici?

- Un grafico mostra **a colpo d'occhio** l'andamento dei dati: crescita, oscillazioni, valori anomali.
- Permette di **confrontare** le misure con un **modello** teorico.
- È il modo standard di **comunicare** un risultato scientifico.
- Il C è ottimo per **calcolare**, ma non ha strumenti grafici semplici.
- Useremo **Python**: lo vedremo a fondo più avanti nel corso, per ora ci basta lo stretto necessario per fare grafici.



Misure di un corpo in caduta libera confrontate con il modello $h(t) = h_0 - \frac{1}{2}gt^2$.

Python in due parole

Python è un linguaggio **interpretato** (ricordate il modulo 6?): niente compilazione, il codice viene eseguito riga per riga.

	C	Python
Esecuzione	`gcc -o prog prog.c` e poi `./prog`	`python3 prog.py`
Tipi	Dichiarati: `double x = 1.5;`	Dedotti dal valore: `x = 1.5`
Fine istruzione	`;`	A capo
Blocchi	`{ }`	Indentazione
Commenti	`//` e `/* */`	`#`
Stampa	`printf("%f\n", x);`	`print(x)`

In Python l'**indentazione** fa parte della sintassi: non è solo una questione di stile come in C.

Interprete e script

Modalità interattiva

Si apre l'interprete con `python3` e si scrivono i comandi dopo il prompt `>>>` :

```
$ python3
>>> x = 3
>>> print(x * 2)
6
>>> exit()
```

Comoda per **provare** al volo.

Script

Si scrivono i comandi in un file `.py` con l'editor…

```
# ciao.py: il primo script
print("Ciao Mondo")
```

…e lo si esegue dal terminale:

```
$ python3 ciao.py
Ciao Mondo
```

Per i grafici useremo **sempre** gli script: si possono modificare e rieseguire.

Le librerie: NumPy e Matplotlib

Python da solo non sa fare grafici: usiamo due **librerie** (moduli) da importare all'inizio dello script.

```
import numpy as np                # calcolo numerico con array
import matplotlib.pyplot as plt   # grafici
```

- `import ... as np`: d'ora in poi la libreria si chiama `np` (un **alias**, per scrivere meno).
- Le funzioni della libreria si usano col punto: `np.linspace( ... )`, `plt.plot( ... )`.
- **NumPy** fornisce gli **array** e le funzioni matematiche (`np.sin`, `np.exp`, `np.sqrt`, `np.pi`, …).
- **Matplotlib** (con il suo modulo `pyplot`) disegna i grafici.
- Il ruolo è simile a `#include <math.h>` in C, ma qui le funzioni si chiamano con il prefisso.

Gli array di NumPy

Un **array** è una sequenza di numeri su cui si può operare **tutta in una volta**.

```python
import numpy as np

x = np.linspace(0, 1, 5)   # 5 punti tra 0 e 1
print(x)
print(x * x)
print(2 * x + 1)
print(np.sin(np.pi * x))
```

```
[0.   0.25 0.5  0.75 1.  ]
[0.     0.0625 0.25   0.5625 1.    ]
[1.  1.5 2.  2.5 3. ]
[0.00000000e+00 7.07106781e-01 1.00000000e+00
 7.07106781e-01 1.22464680e-16]
```

- `np.linspace(a, b, n)`: `n` punti equispaziati da `a` a `b` (**estremi inclusi**).
- Le operazioni agiscono **elemento per elemento**: niente cicli!

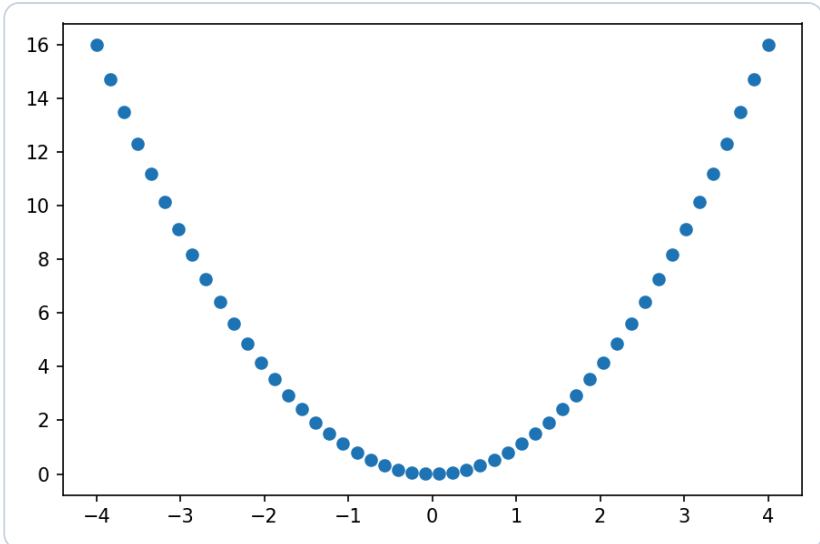
Avete notato `1.22e-16` invece di `0`? È $\sin(\pi)$ calcolato in **virgola mobile**: π non è rappresentabile esattamente (modulo 4).

Il primo grafico

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(-4, 4, 50)
y = x * x

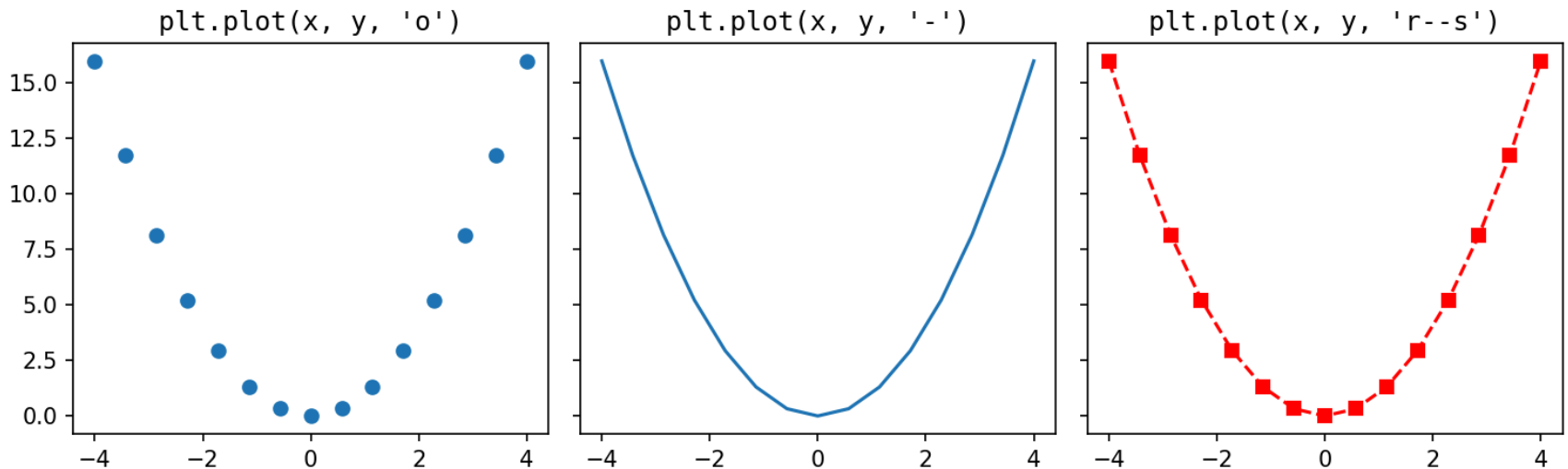
plt.plot(x, y, 'o')
plt.show()
```



- Si calcolano **due array** della stessa lunghezza: ascisse e ordinate.
- `plt.plot(x, y, 'o')` disegna i punti (x_i, y_i) come cerchietti.
- `plt.show()` apre la **finestra** con il grafico (lo script si ferma finché non la chiudete).

Punti o linee? Le stringhe di formato

Il terzo argomento di `plt.plot` è una stringa che sceglie **colore**, **marker** e **linea**.



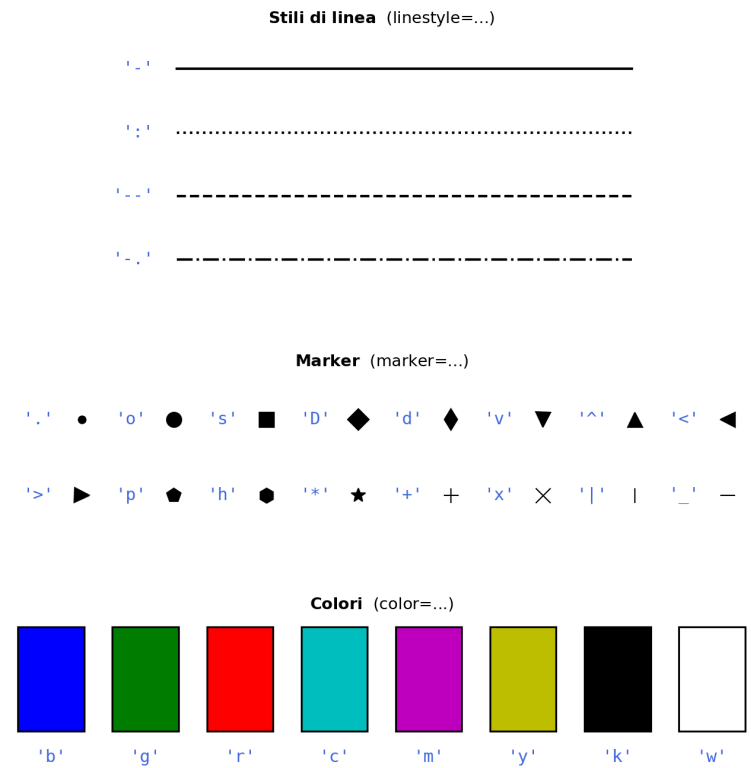
- `'o'` : solo punti (cerchietti) · `'-'` : solo linea continua · `'r--s'` : rosso (`r`), tratteggiato (`--`), quadrati (`s`).
- Regola pratica: **punti** per i dati misurati, **linee** per le funzioni teoriche.

Personalizzare lo stile

Al posto della stringa di formato si possono usare **argomenti con nome**, più leggibili:

```
plt.plot(x, y,
         color='b',        # colore
         linestyle='-',    # tipo di linea
         linewidth=2,      # spessore della linea
         marker='s',       # forma del marker
         markersize=6)     # dimensione del marker
```

- L'ordine degli argomenti con nome **non conta**.
- Quelli non specificati prendono un valore **di default**.
- Oltre alle lettere, i colori si possono indicare per nome (`'orange'`) o in esadecimale (`'#f48c06'`, vi ricorda il modulo 2?).



Leggere i dati da un file

```
#x  #y
-4  16
-3   9
-2   4
-1   1
 0   0
 ...
```

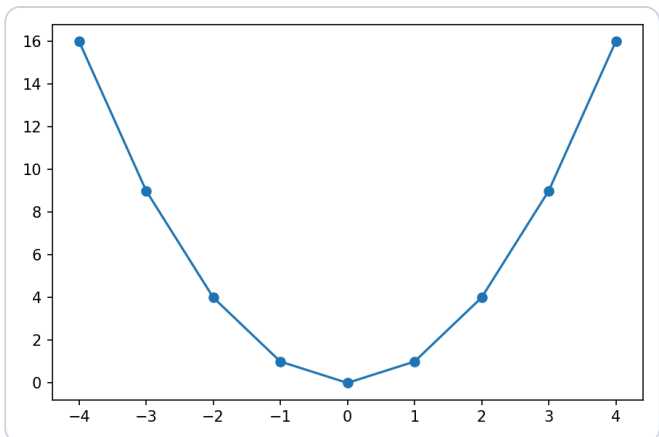
`dati.dat` : colonne separate da spazi; le righe che iniziano con `#` sono commenti.

```python
import matplotlib.pyplot as plt
import numpy as np

x, y = np.loadtxt('dati.dat', unpack=True)

plt.plot(x, y, '-o')
plt.show()
```

- `np.loadtxt` legge un file di numeri **in colonna**.
- `unpack=True` restituisce **una variabile per colonna**: qui `x` e `y` .
- Con `usecols=(0, 2)` si scelgono solo alcune colonne (si conta da **0**).



Un grafico come si deve

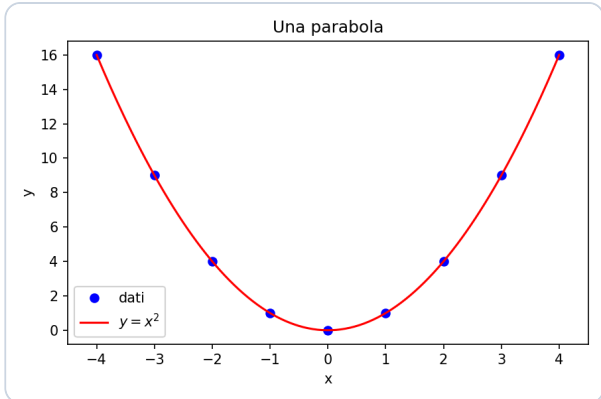
```python
import matplotlib.pyplot as plt
import numpy as np

x, y = np.loadtxt('dati.dat', unpack=True)
xm = np.linspace(-4, 4, 200)

plt.plot(x, y, 'o', color='b', label='dati')
plt.plot(xm, xm**2, '-', color='r', label='$y = x^2$')

plt.title('Una parabola')
plt.xlabel('x')
plt.ylabel('y')
plt.xlim(-4.5, 4.5)
plt.legend()

plt.savefig('parabola.png')
plt.show()
```



- Più `plt.plot` di seguito: le curve finiscono **nello stesso grafico**.
- `label` + `plt.legend()`: la **legenda**. Tra `$ ... $` si può scrivere in LaTeX.
- `plt.xlim` / `plt.ylim`: intervallo degli assi.

Salvare il grafico

```
plt.savefig('grafico.png')   # immagine
plt.savefig('grafico.pdf')   # vettoriale
plt.show()
```

- Il **formato** si sceglie con l'estensione del nome del file.
- `.png` va bene per lo schermo; `.pdf` resta nitido a qualunque ingrandimento (ideale per le relazioni).

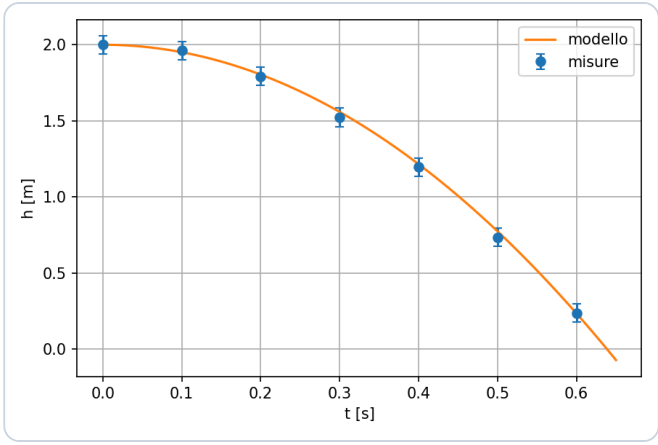
Attenzione all'ordine! `plt.savefig` va chiamato **prima** di `plt.show`. Quando chiudete la finestra il grafico viene cancellato: un `savefig` scritto dopo salva un'immagine **vuota**.

Dati sperimentali: le barre d'errore

```
# t[s]  h[m]   dh[m]
  0.00  2.000  0.06
  0.10  1.963  0.06
  0.20  1.793  0.06
   ...
```

```
t, h, dh = np.loadtxt('caduta.dat', unpack=True)
tm = np.linspace(0, 0.65, 200)

plt.errorbar(t, h, yerr=dh, fmt='o',
             capsize=3, label='misure')
plt.plot(tm, 2.0 - 0.5 * 9.81 * tm**2,
         label='modello')
plt.xlabel('t [s]')
plt.ylabel('h [m]')
plt.grid(True)
plt.legend()
plt.show()
```



- `plt.errorbar` : come `plot` , ma con l'incertezza `yerr` (anche `xerr`).
- `capsize` : i "cappellini" alle estremità delle barre.
- Le **unità di misura** vanno sempre nelle etichette degli assi!

Errori comuni

Cosa succede	Messaggio di Python	Causa
File non trovato	`FileNotFoundError: dato.dat not found.`	Nome sbagliato o cartella sbagliata (`ls` , `pwd`)
Array di lunghezza diversa	`ValueError: x and y must have same first dimension, but have shapes (3,) and (2,)`	`x` e `y` non hanno lo stesso numero di elementi
Righe con colonne diverse	`ValueError: the number of columns changed from 2 to 3 at row 2`	File di dati malformato
Legenda vuota	`UserWarning: No artists with labels found to put in legend.`	Manca `label= ...` in `plt.plot`
Maiuscole	`AttributeError: module 'matplotlib.pyplot' has no attribute 'Show'`	Python distingue maiuscole e minuscole
Immagine bianca	*nessun messaggio*	`plt.savefig` chiamato dopo `plt.show`

Checklist di un buon grafico

- **Assi etichettati**, con le **unità di misura**: `t [s]`, `h [m]`.
- **Titolo** o didascalia che dica cosa si sta guardando.
- **Legenda** se ci sono più curve.
- **Punti** (con barre d'errore) per le misure, **linee** per i modelli.
- **Intervallo degli assi** scelto in modo da mostrare ciò che interessa.
- Curve teoriche con **abbastanza punti** da sembrare lisce (200 punti, non 5).
- Salvate in un **file** (`.png` o `.pdf`), non uno screenshot.